



Урок 9 > Подсистемы для поиска

> Реализация автодополнения

Для начала в системе необходимо реализовать сбор статистики по тем или иным запросам.

Статистику можно собирать в независимые key-value хранилища и затем агрегировать (map-reduce).

В таблице есть колонка запросов и количество раз, которые они встречаются в логах:

Query	Count
Карпов курсы	147
Карпов курсы отзывы	42
Карпов курочка	23
Карпов курган	16
Карпов кубань	15

Затем при прямолинейном подходе на каждую новую букву мы выполняем запрос вида:

```
1 SELECT * FROM stats
2 WHERE query LIKE 'prefix%'
3 ORDER BY count DESC
4 LIMIT 3
```

Минусы у такого подхода:

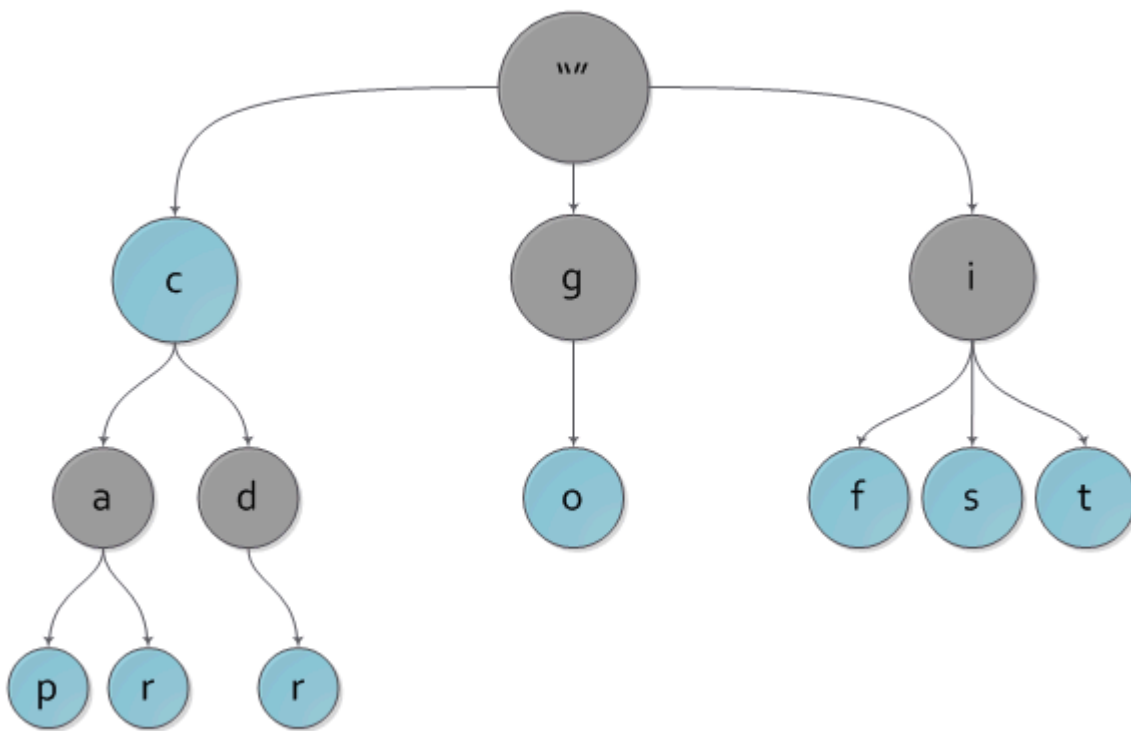
- на каждую введенную букву мы посылаем отдельный запрос;

- каждый раз сканируем всю таблицу, а это $O(N)$ или $O(\log N)$, если построен индекс.

> Префиксное дерево

Префиксное дерево (**бор** или **trie**) — структура данных для компактного хранения строк, устроенная в виде дерева, где на рёбрах между вершинами записаны символы, а некоторые вершины помечены терминальными. Говорят, что префиксное дерево принимает строку S , если существует такая терминальная вершина v , что, если выписать подряд все буквы на путях от корня до v , то получится строка S .

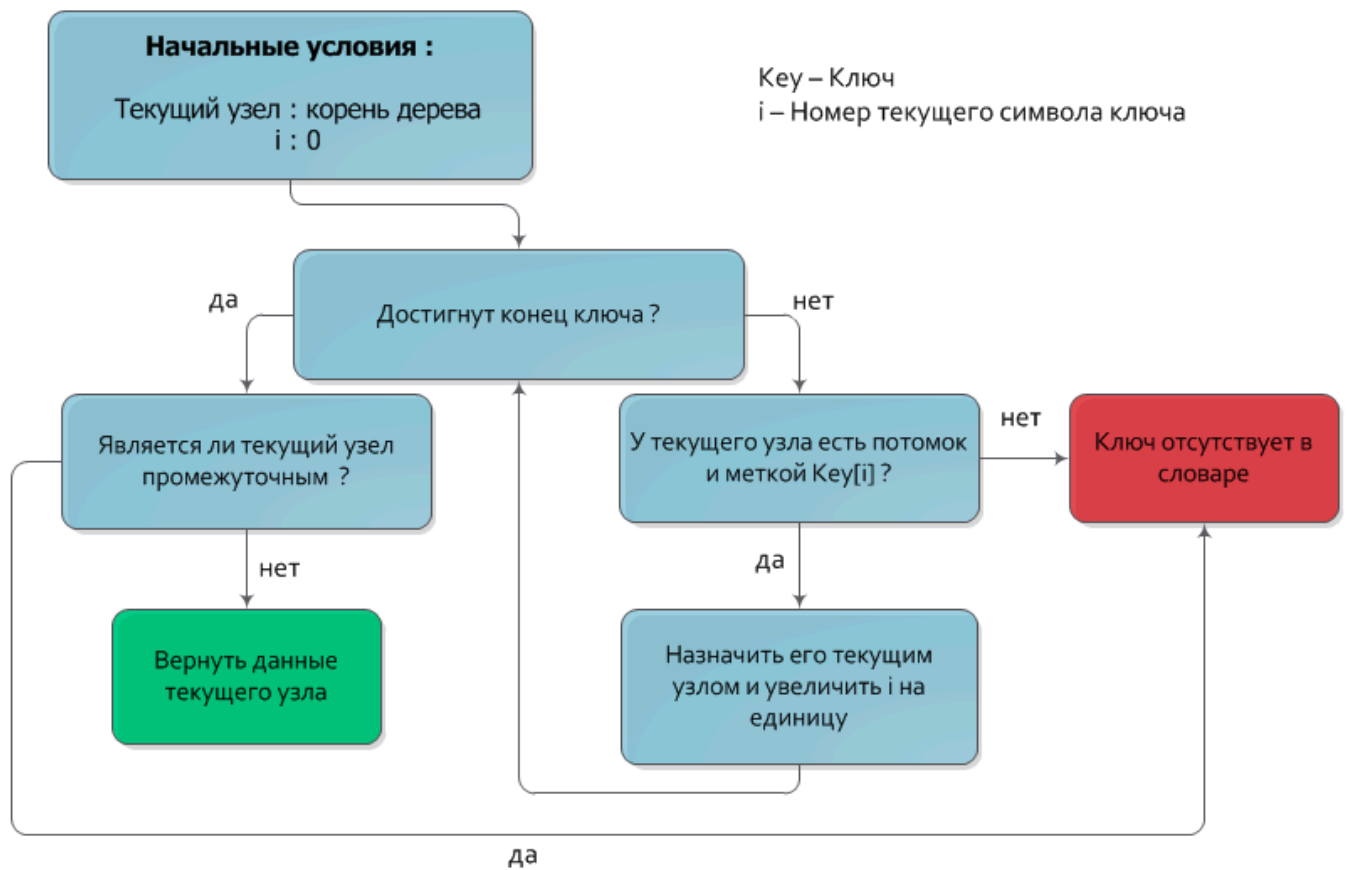
Нагруженное дерево отличается от обычных n -арных деревьев тем, что в его узлах не хранятся ключи. Вместо них в узлах хранятся односимвольные метки, а ключём, который соответствует некоему узлу, является путь от корня дерева до этого узла, а точнее строка, составленная из меток узлов, повстречавшихся на этом пути. В таком случае корень дерева, очевидно, соответствует пустому ключу.



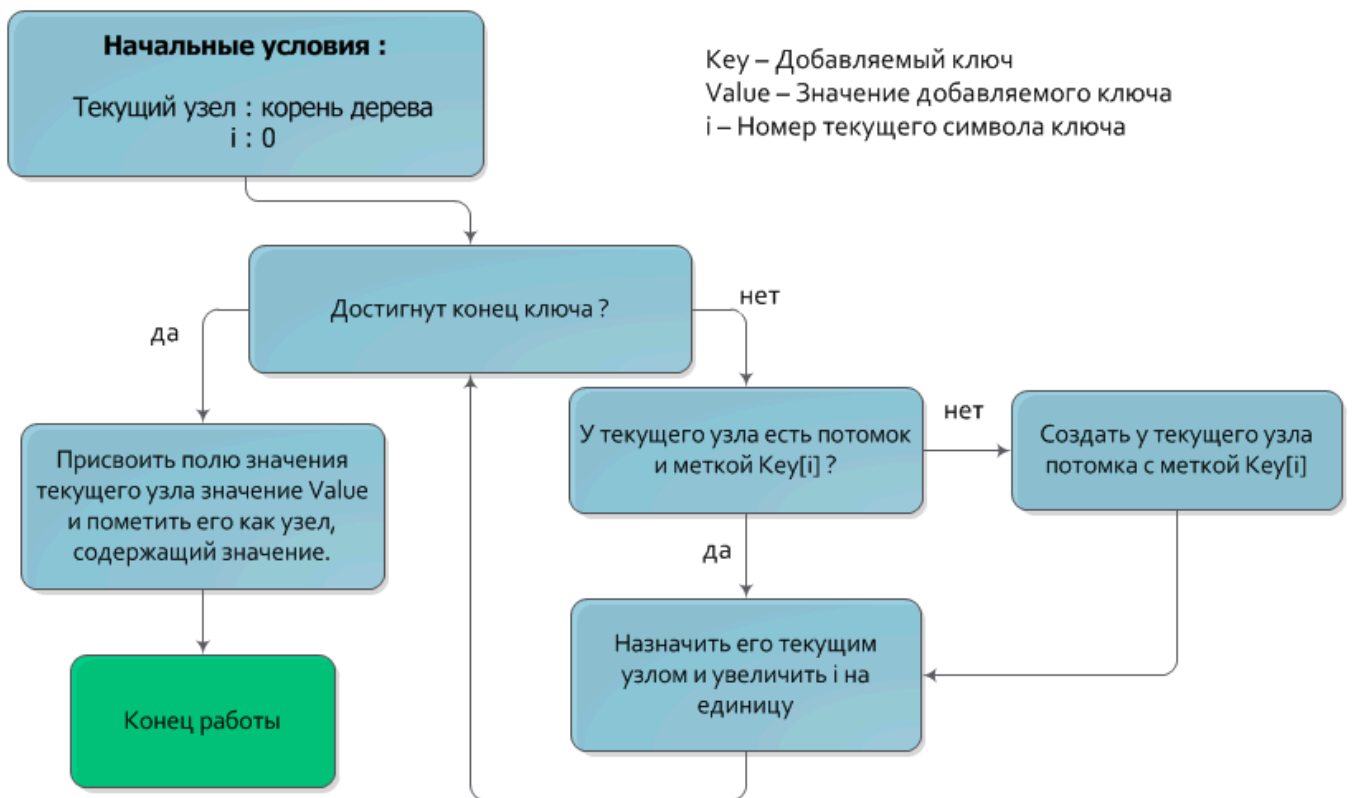
На рисунке вы можете наблюдать пример нагруженного дерева с ключами c , car , car , cdr , go , if , is , it

Основные операции над префиксным деревом

1. Поиск по ключу. Временная сложность этого алгоритма равна $O(|Key|)$.



2. Вставка. Временная сложность добавления ключа — $O(|Key|)$.



3. Удаление. Временная оценка алгоритма удаления — $O(|Key|)$.

Оптимизация

Существует два основных типа оптимизации префиксного дерева:

- **Сжатие.** Сжатое нагруженное дерево получается из обычного удалением промежуточных узлов, которые ведут к единственному не промежуточному узлу. Например, цепочка промежуточных узлов с метками $*a, b, c*$ заменяется на один узел с меткой $*abc.*$
- **Patricia.** Patricia нагруженное дерево получается из сжатого (или обычного) удалением промежуточных узлов, которые имеют одного ребенка.

> Поиск подстроки в строке

Для поиска подстроки в строке можно применить наивный подход.

Будем прикладывать искомую подстроку к каждому возможному месту в строке.

Плюс: простота реализации.

Минус: при частичных совпадениях сложность возрастает до $O(|\text{haystack}| \times |\text{needle}|)$.

> Префикс-функция

Префикс-функцией от строки S называется массив p , где $p(i)$ равно длине самого большого префикса строки $s(0)s(1)s(2) \dots s(i)$, который также является и суффиксом i -того префикса (не считая весь i -й префикс).

Для строки 'abcdabcababcdab' вычисление будет таким:

```
1 1 S[1]='a', k=π=0;
2 2 S[2]='b'!=S[k+1] => k=π=0;
3 3 S[3]='c'!=S[1] => k=π=0;
4 4 S[4]='d'!=S[1] => k=π=0;
5 5 S[5]='a'==S[1] => k=π=1;
6 6 S[6]='b'==S[2] => k=π=2;
7 7 S[7]='c'==S[3] => k=π=3;
8 8 S[8]='a'!=S[4] => k:=π(S, 3)=0, S[8]==S[1] => k=π=1;
9 9 S[9]='b'==S[2] => k=π=2;
10 10 S[10]='c'==S[3] => k=π=3;
11 11 S[11]='d'==S[4] => k=π=4;
12 12 S[12]='a'==S[5] => k=π=5;
13 13 S[13]='b'==S[6] => k=π=6;
14 14 S[14]='c'==S[7] => k=π=7;
15 15 S[15]='d'!=S[8] => k:=π(S, 7)=3, S[15]==S[4] => k=π=4;
16 16 S[16]='a'==S[5] => k=π=5;
```

```
17 17 S[17]='b'==S[6] => k=π=6;
```

И результат таков: `[0,0,0,0,1,2,3,1,2,3,4,5,6,7,4,5,6]` .

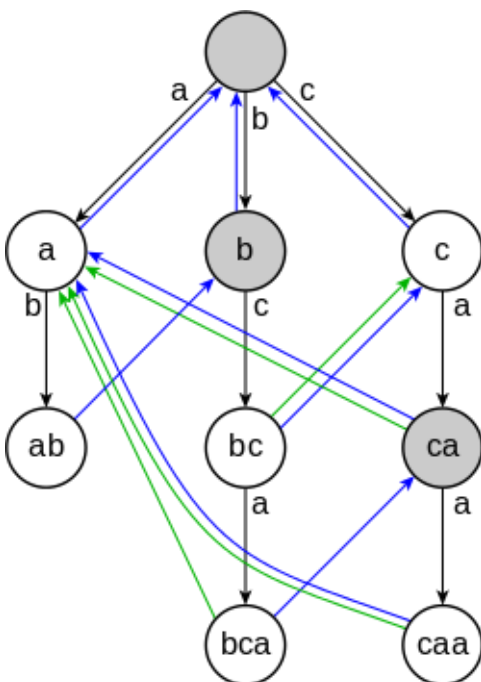
Реализация алгоритма на python:

```
1 def prefix(s):
2     p = [0] * len(s)
3     for i in range(1, len(s)):
4         k = p[i - 1]
5         while k > 0 and s[k] != s[i]:
6             k = p[k - 1]
7         if s[k] == s[i]:
8             k += 1
9         p[i] = k
10    return p
```

> Алгоритм Ахо-Корасик

Алгоритм Ахо-Корасик — алгоритм поиска подстроки, разработанный Альфредом Ахо и Маргарет Корасик в 1975 году, реализует поиск множества подстрок из словаря в данной строке.

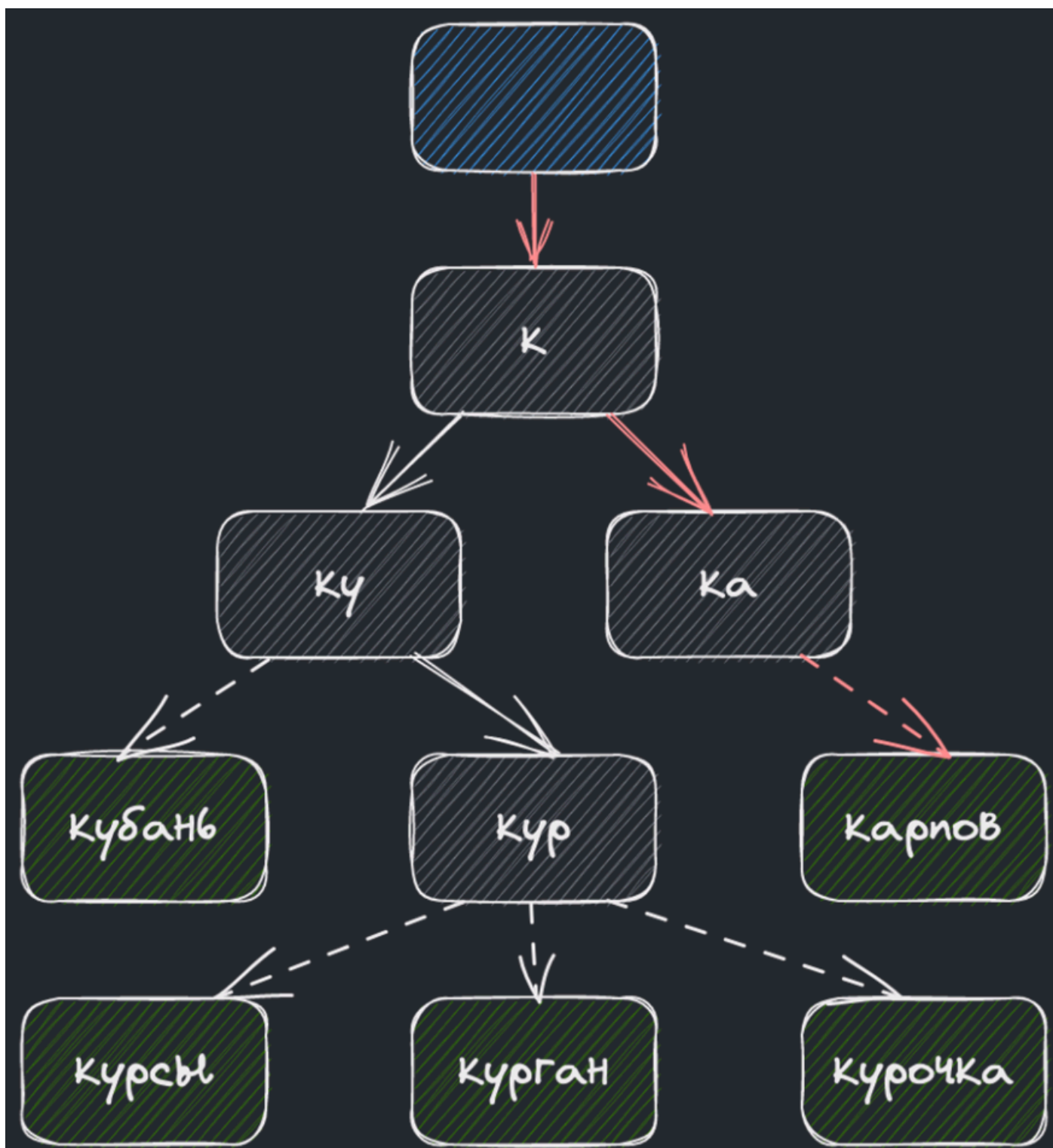
Алгоритм строит конечный автомат, которому затем передаёт строку поиска. Автомат получает по очереди все символы строки и переходит по соответствующим рёбрам. Если автомат пришёл в конечное состояние, соответствующая строка словаря присутствует в строке поиска.



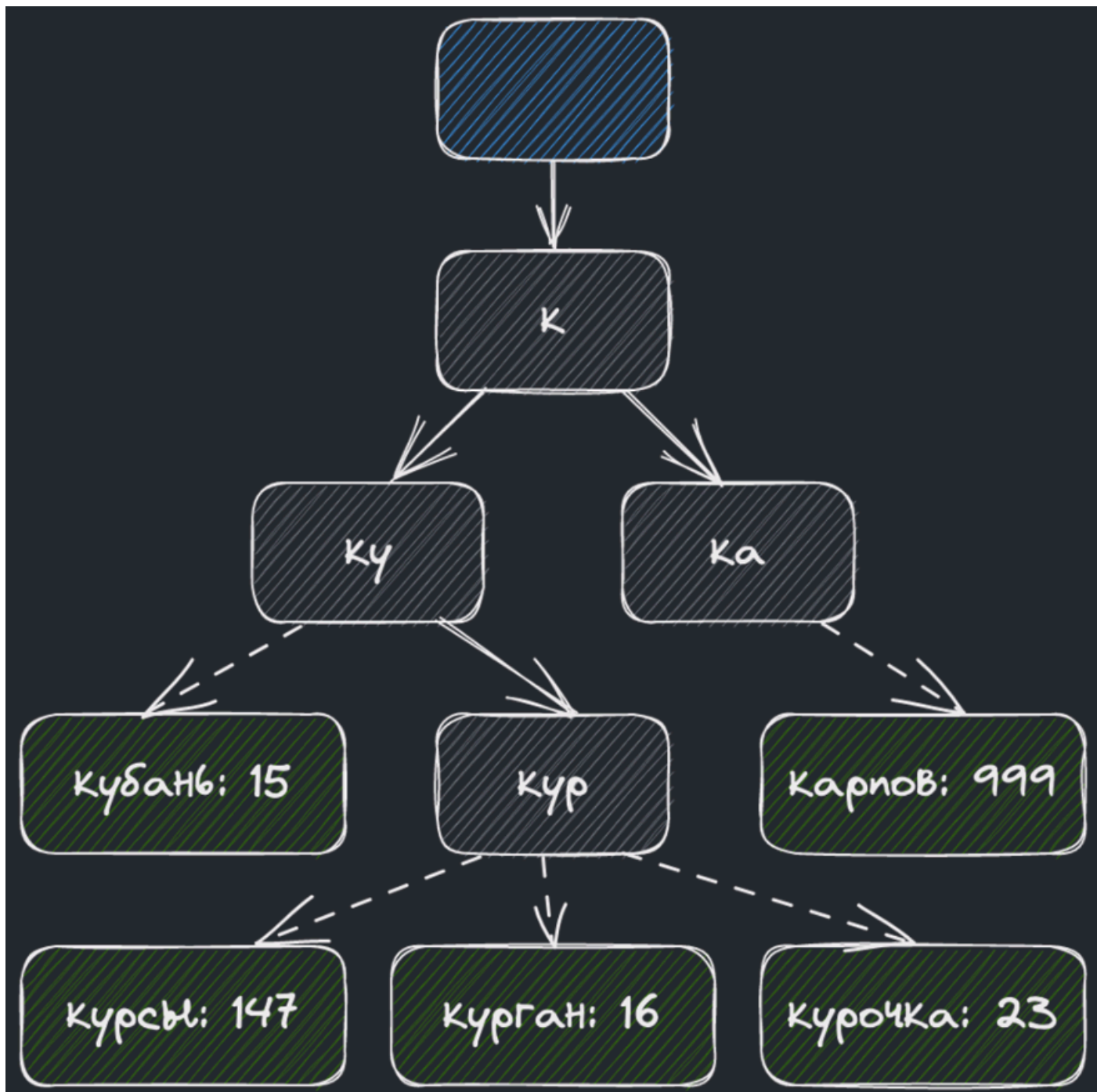
Недетерминированный автомат для словаря {a, ab, bc, bca, c, caa}. Серые вершины промежуточные, белые конечные. Синие стрелки - суффиксные ссылки, зелёные - конечные.

> Автодополнение строк

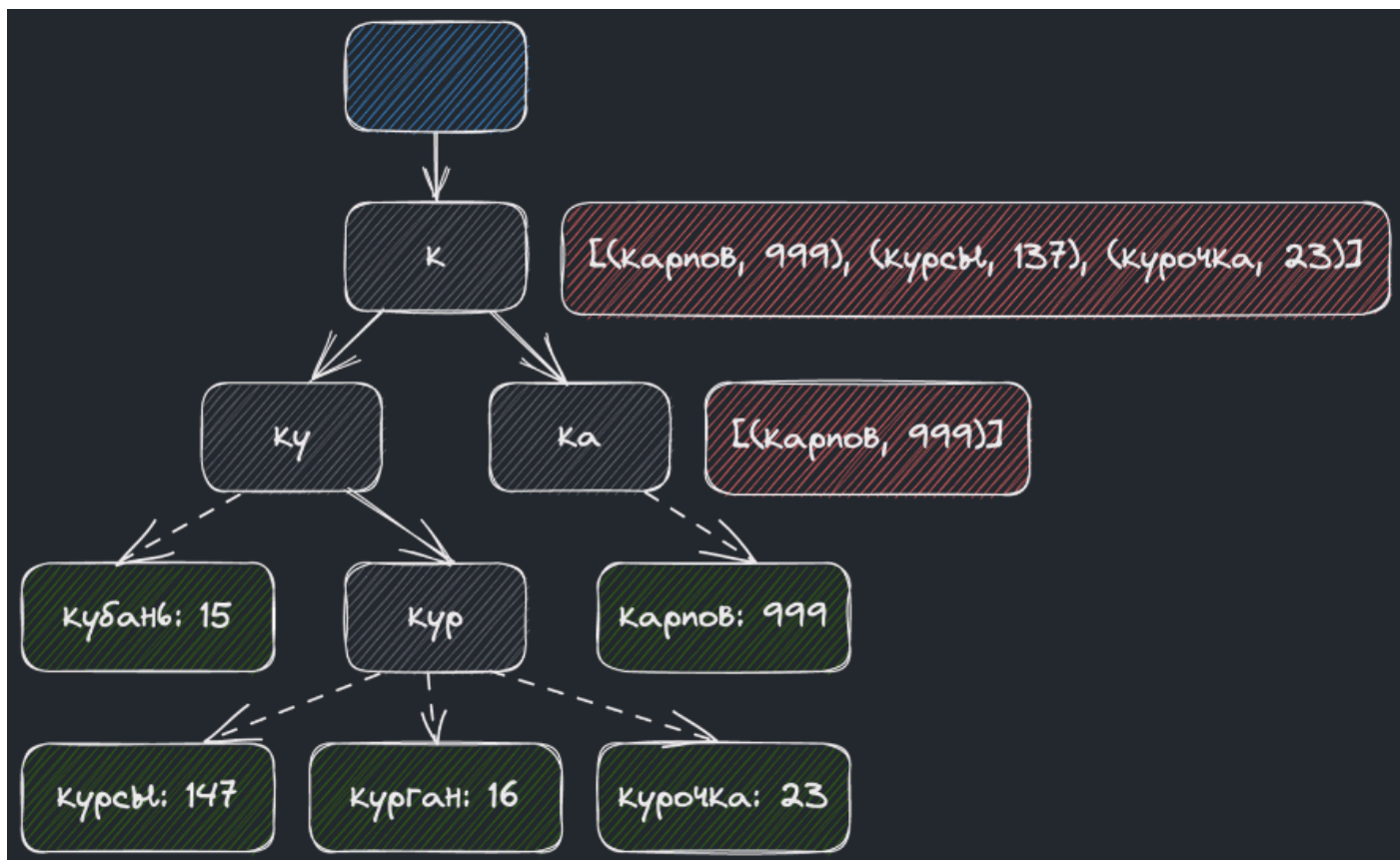
Для оптимального хранения статистики со строками и поиска рассмотрим структуру данных «префиксное дерево». Каждому слову из истории вызовов соответствует путь от корня к листевой вершине дерева.



Статистику по запросам можно хранить в самих вершинах, передвигая счётчик каждого слова, по соответствующему запросу.



Для большей оптимизации можно класть массив статистики запросов в каждой вершине. Это даст возможность получить весь список за константное время.



> Текстовый поиск

Для поиска подстроки в строке отлично подходит префикс-функция.

Для начала соединим нашу подстроку, которую нужно найти в строке, со строкой через разделяющий спецсимвол, который не входит в алфавит. Например можно взять #.



Подстрока найдена, если в значениях функции есть длина подстроки.

> Поиск вхождений по словарю. Реализация

Суть задачи:

Необходимо в заданном тексте найти все вхождения любого слова из заранее подготовленного словаря.

Алгоритм решения:

Для оптимального решения этой задачи подойдет алгоритм Ахо-Корасик.

По словарю строится конечный автомат, а затем через этот автомат прогоняется заданный текст.

> Неточный поиск (wildcard matching)

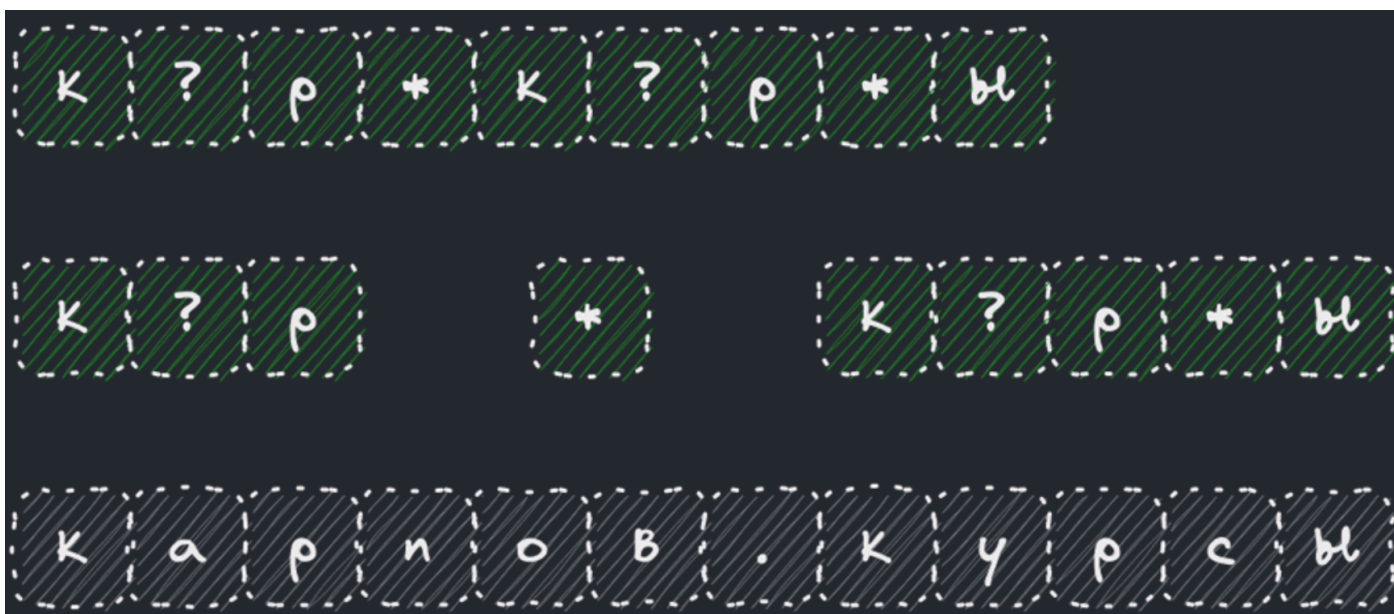
Суть задачи:

Допустим, мы хотим найти шаблон p в строке s , где в шаблоне могут быть символы $?$ и $*$.

Символ $?$ соответствует одной любой букве, тогда как $*$ соответствует любому количеству букв.

Можно считать, что строка s полностью определяется шаблоном p (иначе берём шаблон $*p*$).

Вид такого поиска часто встречается в текстовых редакторах или в сервисах бронирования билетов.



Для оптимального решения задачи воспользуемся динамическим программированием. Построим матрицу

D размера $S \times P$, где $D[m][n]$ – матч между $S[1..m]$ и $P[1..n]$.

Значение $D[m][n]$ легко рассчитать на основе $D[m-1][n-1]$:

- $D[m][n] = D[m-1][n-1]$, если $S[m]$ совпадает с $P[n]$ или $P[n] = “?”$.
- Если $P[n] = “*”$, то опять же $D[m][n]$ будет матчем в след за $D[m-1][n-1]$, при этом и все $D[m+i][n]$ тоже.

ы	0	0	0	0	0	0	0	0	0	0	0	0	1
*	0	0	0	0	0	0	0	0	0	0	0	1	1
р	0	0	0	0	0	0	0	0	0	0	1	0	0
?	0	0	0	0	0	0	0	0	0	1	0	0	0
к	0	0	0	0	0	0	0	0	1	0	0	0	0
*	0	0	0	0	1	1	1	1	1	1	1	1	1
р	0	0	0	1	0	0	0	0	0	0	0	0	0
?	0	0	1	0	0	0	0	0	0	0	0	0	0
к	0	1	0	0	0	0	0	0	0	0	0	0	0
#	1	0	0	0	0	0	0	0	0	0	0	0	0
#	к	а	р	н	о	в	.	к	у	р	с	ы	

> Готовый движок для поиска

Для имплементации поиска можно также воспользоваться готовыми движками.



Например, Apache Lucene представляет из себя движок на Java с биндингами для Python и позволяет:

- Создавать индекс для поиска в документах с текстовыми и строковыми полями — выполнять запросы с определённым для движка синтаксисом;
- Искать вхождения слов и запросов в полях (“title:курс”);
- Добавлять простые логические операции в запрос;
- Находить вхождения на расстоянии друг от друга (“karpovdesign”~4);
- Задавать диапазоны для значений (start_date:[20220601 TO 20220704]).

> Поиск по геолокации

Такой поиск нужен в сервисах, зависящих от локации. Например, когда нам нужно найти ближайший ресторан, курьера или водителя такси.

Restaraunt	Latitude	Longitude
Ayaspasa Rus	41.03663	28.98999
Old Metekhi	41.69002	44.81281
Varazi	41.71131	44.78020
Cafe Kale	41.69252	44.80694
Junior's	40.75830	-73.98670

Таблица с координатами заведений

Если пользователь находится в точке 41.7167, 44.7853, то заведения в радиусе R (почти) надо найти запросом.

```

1 SELECT * FROM restaurants
2 WHERE (latitude BETWEEN 41.7167 - R AND 41.7167 + R) AND
3       (longitude BETWEEN 44.7853 - R and 44.7853 + R)

```

Проблема такого подхода — в необходимости сканировать всю таблицу с заведениями целиком.

Можно ускорить поиск за счёт индексирования, но только в одном измерении. Полученные «полосы» всё равно придётся дорого JOIN’ить.

Для того, чтобы уменьшить зону поиска, карту можно равномерно разбить на квадраты.

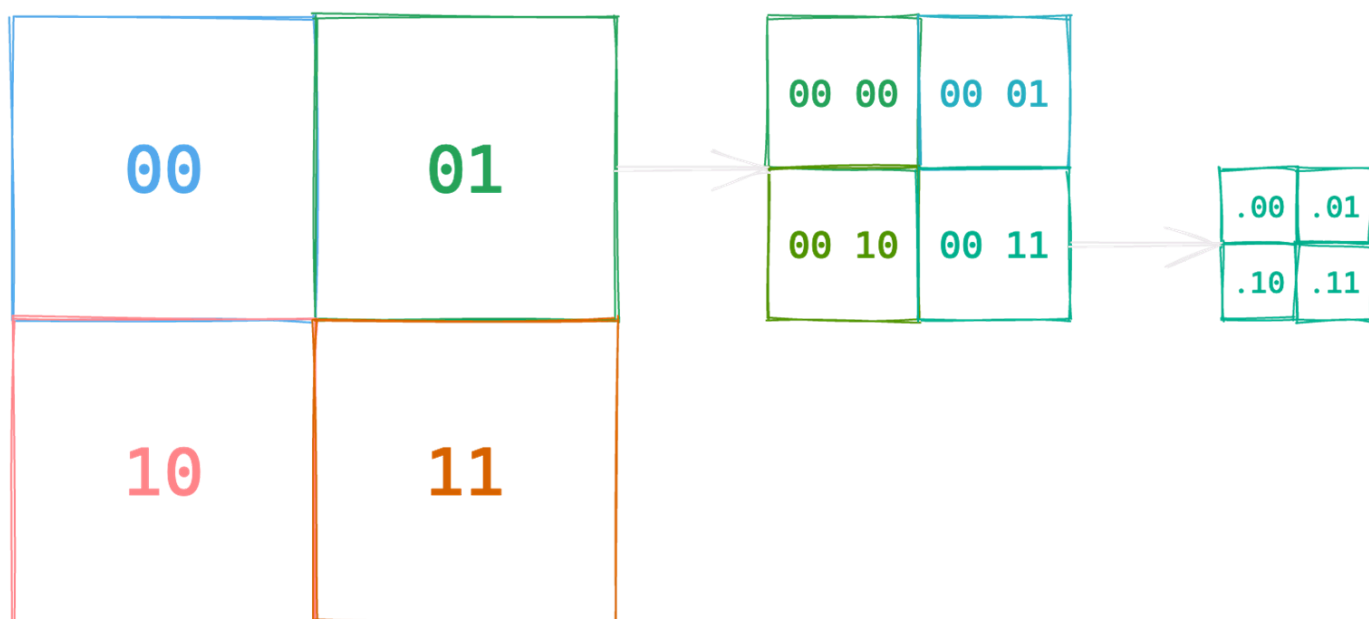


Плюс: ищем только в нужном квадрате (или соседнем).

Минус: точки интереса распределены неравномерно — в одном квадрате Нью-Йорк, в другом — океан.

> GeoHash

Для поиска точки на карте или набора каких-то точек, оптимально разбить карту на отдельные квадраты, каждому из которых задавать адрес. При этом большие квадраты разбиваются на более мелкие с постепенным увеличением адреса последних.



Такой подход позволит записать в 12 бит квадрат со стороной в 1 км, а в 20 бит — около метра

Итоговый адрес ячейки записывается в виде строки. Чтобы найти точки поблизости, достаточно

изучить ячейки с общим префиксом.

Особенности использования GeoHash для поиска ближайших локаций:

- Если префикс у двух точек совпадает до какой-то степени, это гарантирует их близость;
- Обратное не верно — достаточно близкие точки могут не иметь общего префикса вообще;
- Найти ближайшие места можно простым `SELECT * FROM places WHERE geohash LIKE 'prefix%' ;`
- Также два заведения совсем рядом могут иметь разный хеш, поэтому стоит включать соседние клетки;
- При необходимости расширить зону поиска достаточно немного уменьшить префикс;
- Можно создавать локальный геохэш для областей присутствия.

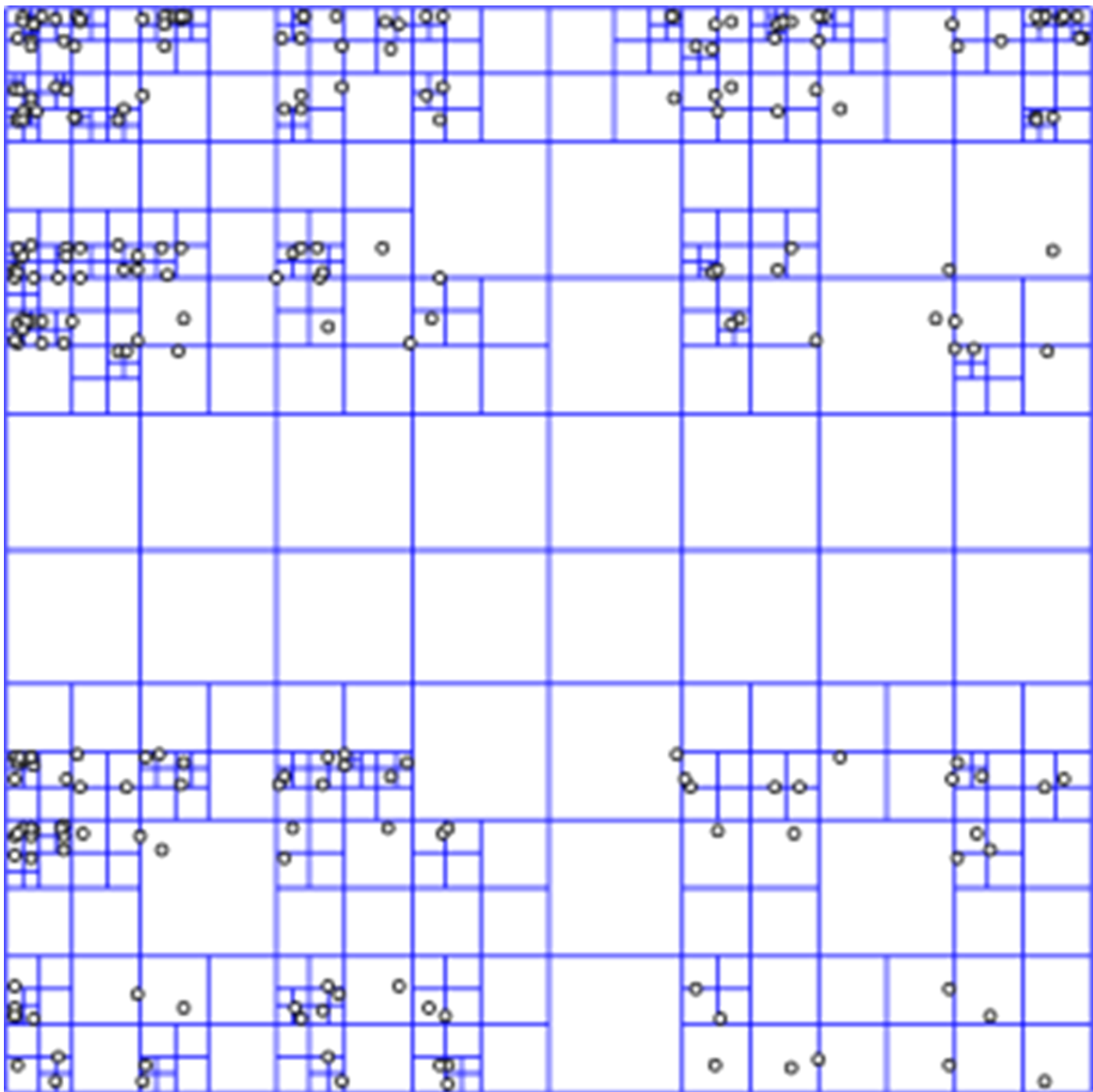
QuadTree

Дерево квадрантов (также **квардродерево**, **4-дерево**, **quadtree**) — дерево, в котором у каждого внутреннего узла ровно 4 потомка. Деревья квадрантов часто используются для рекурсивного разбиения двухмерного пространства по 4 квадранта (области). Области представляют собой квадраты, прямоугольники или имеют произвольную форму. Англоязычный термин **quadtree** был придуман Рафаэлем Финкелем и Джоном Бентли в 1974 году. Аналогичное разбиение пространства известно как *Q*-дерево.

Общие черты разных видов деревьев квадрантов:

- разбиение пространства на адаптирующиеся ячейки (adaptable cells);
- максимально возможный объём каждой ячейки;
- соответствие направления дерева пространственному разбиению.

Предлагаемый подход также разбивает карту на квадраты, но делает это более оптимально — динамически увеличивает глубину разбиения в зависимости от количества точек



Особенности использования QuadTree для поиска ближайших локаций:

- Построение дерева линейно по количеству точек интереса;
- Соответствие между искомой позицией и листом в дереве находится за считанные шаги;
- Для поиска ближайших точек интереса либо останавливаемся в листе, либо поднимаемся выше;
- Два заведения совсем рядом могут попасть в разные ячейки, поэтому можно взять соседние;
- При необходимости расширить зону поиска достаточно подняться на один уровень выше;

- Расширение зоны поиска происходит в неравной степени по сторонам света и по диагонали;
- Все решение легко умещается в оперативную память и поиск проходит стремительно.

> Сравнение двух подходов

Особенности использования геохеширования:

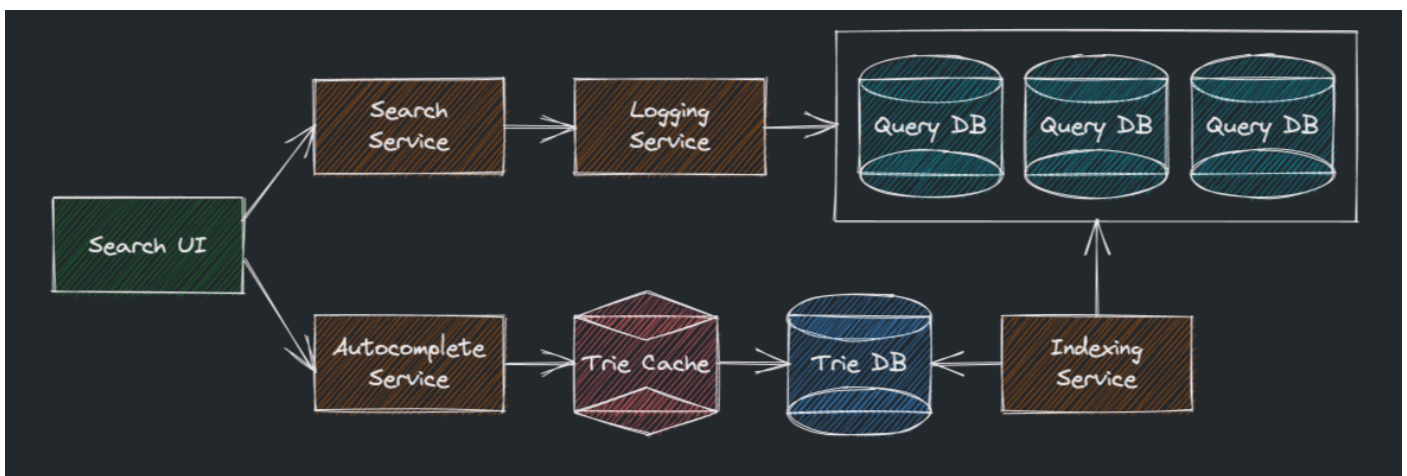
- легко реализовать, не надо ничего «строить»;
- легко найти заведения (точки интереса) в определённом радиусе;
- нельзя адаптировать гранулярность сетки, только перестраивать заново;
- для добавления/удаления записи в индексе достаточно добавить/удалить одну строчку в таблице.

Особенности использования квадродерева:

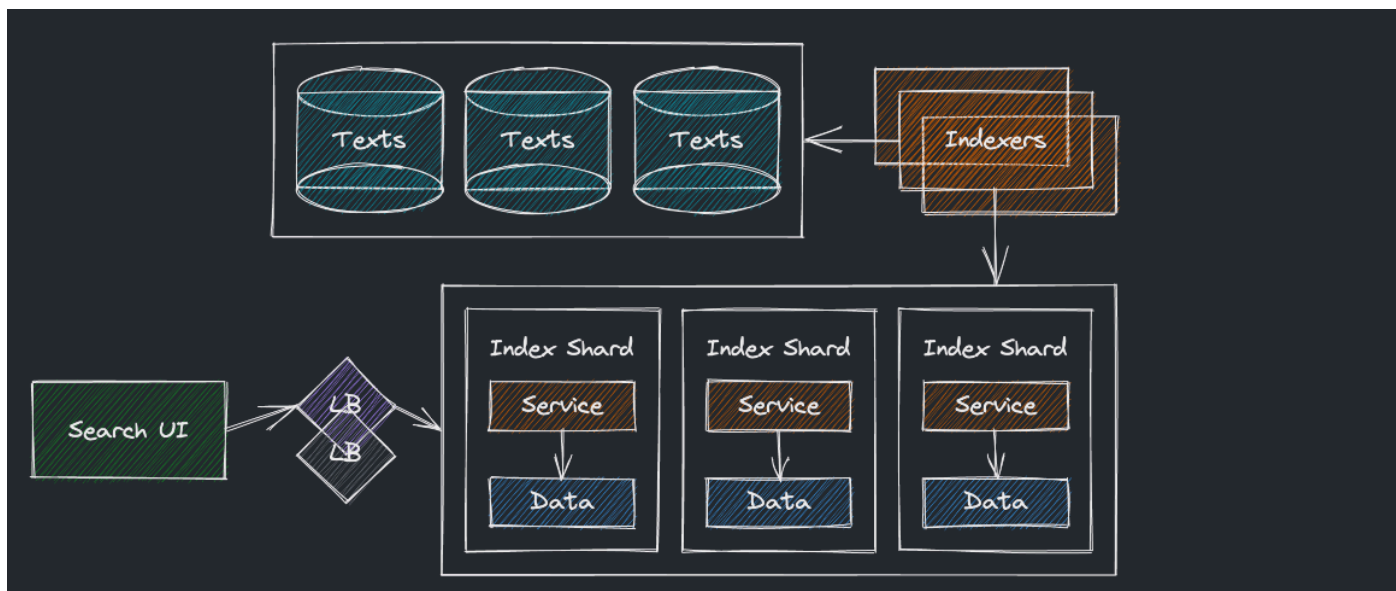
- чуть сложнее реализовать, т.к. нужно строить дерево поиска;
- легко найти top-k ближайших точек интереса;
- можно адаптировать глубину тех или иных участков дерева со временем;
- добавление/удаление записей включает спуск по дереву и балансировку при необходимости.

> Архитектура подсистем для поиска

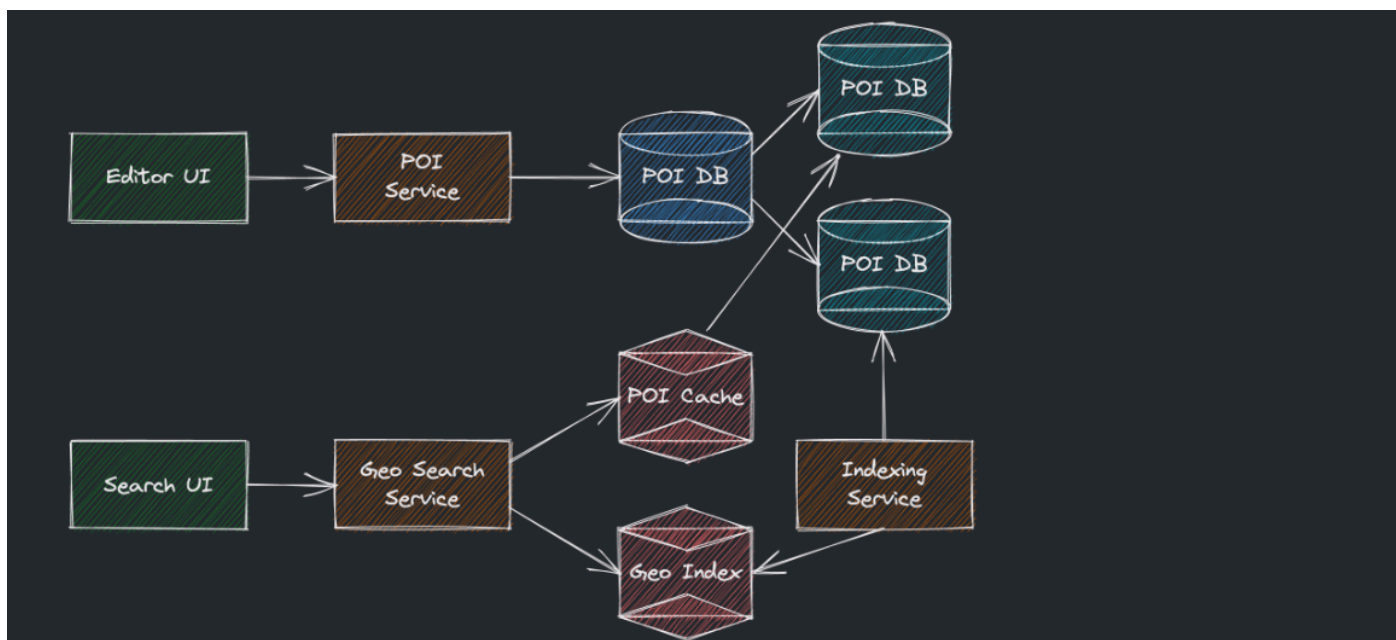
Подсистема для сбора данных, индексации и предоставления API для автодополнения:



Подсистема для индексации и предоставления API для текстового поиска:



Подсистема для индексации и предоставления API для поиска по геолокации:



> Итоги

- Популярными сценариями поиска являются автодополнение, поиск по тексту и геолокации;
- Для автодополнения есть готовый алгоритм, производящий выдачу топа за константное время;
- Для поиска по тексту есть как готовые алгоритмы для простых случаев, так и поисковые движки;
- Простая фильтрация по координатам для поиска точек в окрестности неэффективна;
- Для поиска по геоданным существуют различные решения на основе хешей или деревьев.

